

SCRIPT PRESENTASI PORTOFOLIO

MeetRecall AI Improvement | Sri Lutfiya Dwiyeeni

PANDUAN PENGGUNAAN SCRIPT

- Script disusun per slide sesuai urutan presentasi (14 slide total).
 - Teks dalam [kurung kotak] adalah petunjuk aksi atau penekanan visual.
 - Perkiraan durasi total: 12 menit. Sesuaikan tempo bicara dengan audiens.
 - Kecepatan bicara ideal: ± 130 –140 kata per menit.
- =====

[SLIDE 1] OPENING / PEMBUKA 🕒 ± 40 detik

Assalamualaikum warahmatullahi wabarakatuh.

Selamat pagi/siang/sore, Bapak/Ibu dan rekan-rekan sekalian.

Perkenalkan, nama saya Sri Lutfiya Dwiyeeni. Hari ini saya akan mempresentasikan proyek portofolio saya di bidang Kecerdasan Buatan — MeetRecall AI, sebuah platform berbasis AI yang dirancang untuk mengolah rekaman rapat secara otomatis, melalui tahapan analisis dan sepenuhnya privat.

[SLIDE 2] TABLE OF CONTENTS 🕒 ± 25 detik

Sebelum memulai, izinkan saya memberikan gambaran singkat tentang alur presentasi hari ini.

Saya akan membahas sepuluh poin utama: dimulai dari profil singkat saya, lalu Executive Summary dan tantangan yang melatarbelakangi proyek ini, kemudian cara kerja AI dan teknologi yang digunakan, dilanjutkan dengan empat demo langsung fitur-fitur utama, dan diakhiri dengan kesimpulan serta rencana pengembangan ke depan.

Mari mulai dari poin pertama pada pembahasan hari ini.

[SLIDE 3] ABOUT ME 🕒 ± 60 detik

Sebelum masuk ke proyek, izinkan saya memperkenalkan diri secara singkat.

Saya Sri Lutfiya Dwiyeeni, lulusan S1 Matematika dari Universitas Diponegoro dengan fokus di bidang Artificial Intelligence, Machine Learning, NLP, LLM, dan Generative AI.

Beberapa proyek yang telah saya kembangkan antara lain:

- [1] Clinical Research RAG Mini Chatbot — chatbot berbasis RAG yang dirancang untuk menjawab pertanyaan seputar topik klinis seperti Nightmare Disorder, Insomnia Related Therapy, dan Major Depressive Episode. Sistem ini mengambil informasi dari dokumen medis secara akurat menggunakan vector search.
- [2] MLOps Model Monitoring — pipeline MLOps end-to-end untuk memprediksi harga rumah. Dilengkapi dengan monitoring performa model menggunakan MLflow, API deployment via FastAPI, dan containerisasi Docker untuk produksi.
- [3] SmartSplit Bill Assistant — aplikasi web yang secara otomatis mengekstrak struk belanja menggunakan OCR dan AI, lalu membagi tagihan antar peserta secara cerdas dan real-time.
- [4] Intelligent Customer Assistant — chatbot layanan pelanggan berbasis RAG dengan FastAPI dan LangChain yang mampu menjawab pertanyaan produk secara akurat, kontekstual, dan bebas halusinasi.

Dan proyek terbaru serta yang paling komprehensif adalah MeetRecall AI yang akan saya bahas hari ini.

[SLIDE 4] EXPERIENCE 🕒 ± 50 detik

Untuk pengalaman profesional, saat ini saya bekerja sebagai Data Scientist di NoLimit Indonesia, di mana saya mengembangkan RAG Chatbot, pipeline ABSA sentiment analysis, dan Agentic AI untuk automated report generation dan mendeplolnya di server SSH menggunakan termius dan command Linux.

Sebelumnya, saya menyelesaikan AIML Bootcamp di Dibimbing.id dan meraih MVP Award serta Best Final Project Award. Di Bangkit Academy 2024, tim saya masuk Top 50 Best Final Project dari 644 tim.

Seluruh pengalaman ini menjadi fondasi dalam membangun MeetRecall AI.

[SLIDE 5] EXECUTIVE SUMMARY 🕒 ± 90 detik

Sekarang mari masuk ke inti presentasi.

MeetRecall AI adalah Meeting Intelligence Platform yang secara otomatis mengubah rekaman rapat apa pun menjadi wawasan berbasis AI, secara privat, tanpa usaha manual sama sekali.

Platform ini memiliki lima kemampuan utama:

- [1] Transkripsi otomatis — sistem mendengarkan rekaman dan mengubah setiap kata menjadi teks lengkap beserta timestamp, tanpa perlu mencatat manual.

[2] Tanya jawab rapat — ketik pertanyaan apa pun dalam bahasa alami, AI menjawab langsung dari transkrip nyata.

[3] Penemuan topik — AI mengorganisir percakapan menjadi tema-tema berlabel secara otomatis.

[4] Analisis emosi pembicara — setiap peserta mendapat breakdown nada: positif, netral, atau negatif.

[5] Sepenuhnya lokal — tidak ada data yang dikirim ke cloud. Privasi terjaga penuh.

[SLIDE 6] THE CHALLENGE 🕒 ± 70 detik

Sebelum masuk ke solusi, penting untuk memahami masalah yang ingin diselesaikan.

Setiap hari, para profesional menghadapi empat tantangan besar dalam mengelola rekaman rapat:

1. Pertama, waktu yang terbuang. Menulis ulang rapat satu jam bisa memakan empat hingga lima jam kerja tambahan.
2. Kedua, sulit menemukan informasi. Untuk tahu apa yang diputuskan, seseorang harus memutar ulang seluruh rekaman.
3. Ketiga, tidak ada gambaran topik. Ketika satu rapat membahas banyak hal, hampir tidak mungkin mengorganisirnya secara manual.
4. Keempat, konteks emosional yang hilang. Tanpa alat yang tepat, kita tidak tahu bagaimana sikap setiap peserta selama diskusi.

MeetRecall AI hadir untuk menjawab keempat tantangan ini sekaligus.

[SLIDE 7] HOW THE AI WORKS 🕒 ± 85 detik

Sekarang izinkan saya menjelaskan bagaimana sistem ini bekerja dari awal hingga akhir.

[Tunjuk step 1] Upload Rekaman — pengguna cukup mengunggah file audio atau video. Sistem mendukung lebih dari sepuluh format berbeda.

[Tunjuk step 2] Speech to Text — AI mendengarkan setiap kata dan menuliskannya secara lengkap beserta timestamp.

[Tunjuk step 3] Speaker Identification — setiap suara dikenali dan diberi label otomatis; kita tahu siapa berbicara kapan.

[Tunjuk step 4] Content Understanding — AI membaca transkrip untuk memahami makna, konteks, dan poin-poin kunci.

Dari pemahaman konten ini, tiga fitur utama tersedia: Ask the Meeting untuk tanya jawab berbasis RAG, Discover Topics untuk pengelompokan tema otomatis, dan Read the Room untuk analisis nada emosi per pembicara.

[SLIDE 8] BUILT WITH 🕒 ± 60 detik

MeetRecall AI dibangun menggunakan tumpukan teknologi modern yang seluruhnya bersifat open-source.

Framework dan tools: Python, FastAPI untuk backend API, Streamlit untuk antarmuka pengguna, Docker untuk containerisasi, dan HuggingFace sebagai sumber model AI.

Model AI yang digunakan end-to-end: Whisper untuk speech-to-text, Pyannote untuk identifikasi pembicara, MiniLM untuk semantic embedding dan pencarian, Qwen2 untuk reasoning dan label topik, serta RoBERTa untuk klasifikasi sentimen per pembicara.

Yang istimewa: semua model berjalan sepenuhnya lokal — tanpa biaya API, tanpa risiko kebocoran data.

[SLIDE 9] DEMO — Upload & Transcription 🕒 ± 45 detik

Mari kita lihat demo fitur pertama: Upload dan Transkripsi.

[Tunjuk tampilan demo] Pengguna cukup drag and drop file rekaman. Sistem menerima lebih dari sepuluh format audio dan video, dengan pilihan provider lokal yang privat atau cloud yang lebih cepat.

Hasilnya adalah transkrip penuh dengan setiap pembicara teridentifikasi jelas dan dilengkapi timestamp. Dalam pengujian: rekaman 24 menit, 7 pembicara, 46 segmen terdeteksi — semuanya otomatis, tanpa intervensi manual.

[SLIDE 10] DEMO — Ask Your Meeting 🕒 ± 45 detik

Fitur kedua adalah Ask Your Meeting — tanya jawab cerdas berbasis RAG, Retrieval-Augmented Generation.

Pengguna cukup mengetik pertanyaan dalam bahasa alami, dan AI menemukan jawaban dari transkrip nyata. Setiap jawaban selalu disertai referensi ke bagian spesifik transkrip — bukan jawaban yang dikarang.

Contoh: "Keputusan apa yang diambil?" atau "Apa yang dibahas Speaker A mengenai anggaran?" Fitur ini bekerja dalam Bahasa Indonesia maupun Bahasa Inggris.

[SLIDE 11] DEMO — Topic Discovery 🕒 ± 40 detik

Fitur ketiga adalah Topic Discovery — penemuan topik otomatis.

AI mengelompokkan seluruh segmen diskusi ke dalam tema-tema bermakna secara otomatis. Topik beserta labelnya dihasilkan sepenuhnya oleh AI, tanpa kategorisasi manual.

Dalam pengujian: dari rekaman 24 menit dengan 46 segmen, AI menemukan 5 topik berbeda. Bayangkan seberapa cepat Anda bisa memahami isi rapat panjang tanpa perlu menontonnya ulang.

[SLIDE 12] DEMO — Read the Room 🕒 ± 40 detik

Fitur keempat dan yang paling unik adalah Read the Room — membaca suasana ruangan.

AI menganalisis bagaimana setiap pembicara tampil sepanjang rapat. Tiap peserta mendapat breakdown emosi: persentase segmen positif, netral, dan negatif.

Dalam pengujian: 7 pembicara dianalisis; suasana rapat didominasi nada netral 59,8%, dengan 26,6% nada positif. Fitur ini berguna untuk team health check, review client call, maupun refleksi pasca-rapat.

[SLIDE 13] CONCLUSION & FUTURE IMPROVEMENTS 🕒 ± 55 detik

Baik, mari bisa saya simpulkan mengenai project ini bahwa:

MeetRecall AI berhasil dikembangkan menjadi platform Meeting Intelligence dengan empat fitur terintegrasi. Pipeline NLP end-to-end — ASR, Diarization, Embeddings, RAG, Clustering, Sentiment — berjalan lokal dengan model open-source, dalam arsitektur modular FastAPI + Streamlit yang dikontainerisasi Docker, mendukung 10+ format file.

Untuk Rencana ke depan: saya akan melakukan Model Optimisation dengan INT8 quantisation, Fine-Tuning Whisper untuk Bahasa Indonesia, Real-time Streaming berbasis WebSocket, dan Speaker Re-ID lintas rapat.

[SLIDE 14] PENUTUP / CLOSING 🕒 ± 35 detik

Demikian presentasi portofolio saya mengenai MeetRecall AI.

MeetRecall AI membuktikan bahwa dengan kombinasi model AI open-source yang tepat, dapat membangun sistem yang mengubah rekaman rapat biasa menjadi wawasan yang kaya dan bisa ditindaklanjuti — sepenuhnya privat, di komputer Anda sendiri.

Saya terbuka untuk diskusi, kolaborasi, maupun peluang di bidang AI dan Machine Learning. Hubungi saya melalui email srilutfiyadwiy@gmail.com, LinkedIn, atau GitHub [rilufiyy](#).

Terima kasih banyak atas waktu dan perhatian Bapak/Ibu. Saya siap menjawab pertanyaan.

Wassalamualaikum warahmatullahi wabarakatuh.

=====

CATATAN PRESENTER

- Berlatihlah minimal 2–3 kali sebelum presentasi sebenarnya.
- Pastikan demo aplikasi sudah berjalan sebelum presentasi dimulai.
- Siapkan jawaban teknis: mengapa memilih model ini, bagaimana akurasi, dll.

ALOKASI WAKTU (Total 12 Menit):

Slide 1	Opening	40 detik
Slide 2	Table of Contents	25 detik
Slide 3	About Me	60 detik
Slide 4	Experience	50 detik
Slide 5	Executive Summary	90 detik
Slide 6	The Challenge	70 detik
Slide 7	How the AI Works	85 detik
Slide 8	Built With	60 detik
Slide 9	Demo – Upload	45 detik
Slide 10	Demo – Ask Your Meeting	45 detik
Slide 11	Demo – Topic Discovery	40 detik
Slide 12	Demo – Read the Room	40 detik
Slide 13	Conclusion	55 detik
Slide 14	Closing	35 detik
TOTAL		720 detik = 12 menit

PERKIRAAN PERTANYAAN & JAWABAN

Sesi Tanya Jawab | MeetRecall AI Presentation

=====

TEKNIS & ARSITEKTUR

TEKNIS

? Kenapa memilih Whisper-small? Kenapa tidak pakai yang lebih besar?

☞ Whisper-small dipilih karena keseimbangan terbaik antara akurasi dan kecepatan untuk berjalan di CPU/GPU lokal. Model yang lebih besar seperti Whisper-large memang lebih akurat, tapi membutuhkan RAM 4–8x lebih besar dan waktu proses jauh lebih lama. Untuk use case meeting transcription, Whisper-small sudah memberikan hasil yang sangat memadai.

TEKNIS

? Berapa akurasi transkripsinya? Apakah sudah diuji?

☞ Dalam pengujian dengan rekaman meeting berbahasa Inggris, Whisper-small menghasilkan Word Error Rate (WER) sekitar 5–10%. Untuk Bahasa Indonesia performanya sedikit lebih rendah karena model belum di-fine-tune khusus untuk Bahasa Indonesia — ini masuk dalam rencana pengembangan ke depan.

TEKNIS

? Bagaimana cara kerja RAG di proyek ini secara teknis?

☞ Setelah transkripsi selesai, setiap segmen percakapan diubah menjadi vektor 384 dimensi menggunakan model paraphrase-multilingual-MiniLM-L12-v2. Vektor-vektor ini diindeks di FAISS. Saat ada pertanyaan masuk, pertanyaan juga diubah ke vektor, lalu FAISS mencari top-k segmen paling relevan (cosine similarity). Segmen-segmen itu menjadi konteks untuk model Qwen2 menghasilkan jawaban yang grounded.

TEKNIS

? Kenapa pakai FAISS? Kenapa tidak pakai vector database seperti Pinecone atau Chroma?

☞ FAISS dipilih karena berjalan sepenuhnya in-memory dan on-disk tanpa infrastruktur tambahan — cocok untuk deployment lokal tanpa cloud dependency. Pinecone atau Chroma memerlukan server terpisah atau koneksi cloud yang bertentangan dengan prinsip privasi lokal proyek ini. Untuk skala meeting data yang relatif kecil, FAISS sudah sangat efisien.

TEKNIS

? Bagaimana speaker diarization bekerja? Seberapa akurat?

☞ Pyannote 3.1 menggunakan neural network untuk mendeteksi kapan setiap pembicara mulai dan berhenti bicara berdasarkan karakteristik suara. Hasilnya berupa segmen waktu berlabel SPEAKER_00, SPEAKER_01, dst. Akurasi sangat bergantung pada kualitas audio — rekaman dengan noise rendah dan pembicara yang tidak saling tumpang tindih menghasilkan diarization yang sangat akurat.

MODEL & KECERDASAN BUATAN

MODEL

? Kenapa memilih Qwen2-0.5B untuk RAG? Modelnya kecil, apakah cukup?

☞ Qwen2-0.5B dipilih karena ini model causal LLM instruction-tuned terkecil yang masih punya kemampuan multilingual solid, termasuk Bahasa Indonesia dan Inggris. Dalam konteks RAG, model tidak perlu "pintar" sendirian — tugasnya hanya merangkai jawaban dari konteks yang sudah disediakan FAISS. Hasilnya terbukti cukup baik untuk use case meeting Q&A.

MODEL

? Apakah sistem bisa salah dalam memberikan jawaban (hallucination)?

☞ Risiko halusinasi diminimalkan karena model RAG ini hanya boleh menjawab berdasarkan konteks dari transkrip nyata yang diberikan. Jika informasi tidak ada di transkrip, sistem idealnya menjawab tidak tahu. Setiap jawaban juga disertai kutipan sumber sehingga pengguna bisa verifikasi langsung ke segmen transkrip aslinya.

MODEL

? Apakah sentiment analysis-nya akurat untuk Bahasa Indonesia?

☞ Model CardiffNLP RoBERTa dilatih pada Twitter data yang bersifat bahasa percakapan, sehingga cukup generalisasi ke teks conversational. Namun untuk Bahasa Indonesia murni, akurasi bisa lebih rendah dibandingkan Bahasa Inggris. Ini juga masuk roadmap pengembangan — fine-tuning model sentiment khusus untuk Bahasa Indonesia.

DEPLOYMENT & PRODUKSI

DEPLOYMENT

? Berapa spesifikasi minimum untuk menjalankan sistem ini?

☞ Untuk berjalan di CPU: minimal 8GB RAM (16GB direkomendasikan), 10GB storage untuk model dan data. Untuk GPU: NVIDIA GPU dengan minimal 4GB VRAM. Proses transkripsi rekaman 24 menit di CPU memakan waktu sekitar 10–15 menit, sedangkan di GPU bisa kurang dari 3 menit.

DEPLOYMENT

? Bisakah sistem ini di-deploy ke cloud atau server produksi?

☞ Ya, sangat bisa. Karena sudah dikontainerisasi dengan Docker Compose, deployment ke cloud seperti AWS EC2, GCP, atau server on-premise tinggal menjalankan docker compose up di server tujuan. Untuk produksi skala besar, backend FastAPI juga bisa di-scale horizontally dengan menambah worker.

DEPLOYMENT

? Berapa lama waktu proses untuk satu rekaman meeting?

☞ Tergantung durasi rekaman dan hardware. Untuk rekaman 24 menit di GPU NVIDIA: transkripsi ~2–3 menit, embedding dan indexing ~30 detik. RAG chat dan sentiment analysis berjalan real-time (< 10 detik per query). Topic clustering tergantung jumlah segmen, umumnya < 1 menit.

PRIVASI & NILAI BISNIS

PRIVASI

? Seberapa aman data meeting yang diproses? Apakah dikirim ke server eksternal?

💬 Semua pemrosesan berjalan 100% lokal di komputer pengguna. Tidak ada data audio, transkrip, atau pertanyaan yang dikirim ke server eksternal — kecuali jika pengguna memilih provider AssemblyAI (cloud). Dengan provider lokal (default), data tidak pernah meninggalkan mesin pengguna.

BISNIS

? Apa yang membedakan MeetRecall AI dari tools sejenis seperti Otter.ai atau Fireflies?

💬 Perbedaan utama ada tiga: pertama, MeetRecall AI berjalan sepenuhnya lokal tanpa biaya subscription. Kedua, open-source dan bisa dikustomisasi sesuai kebutuhan. Ketiga, memiliki fitur analitik lebih dalam seperti topic clustering dan sentiment analysis per pembicara yang tidak umum ada di tools komersial tingkat dasar.

BISNIS

? Apakah proyek ini bisa dikembangkan menjadi produk komersial?

💬 Sangat berpotensi. Beberapa arah komersialisasi yang bisa dijajaki: SaaS dengan model freemium, white-label solution untuk perusahaan, atau integrasi sebagai plugin ke platform meeting seperti Zoom atau Google Meet. Fondasi teknis sudah production-ready dengan Docker dan REST API yang terdokumentasi.

PENGEMBANGAN & MASA DEPAN

PENGEMBANGAN

? Apa tantangan terbesar dalam membangun proyek ini?

💬 Tantangan terbesar ada dua: pertama, mengintegrasikan pipeline Whisper + Pyannote secara akurat — alignment timestamp antara ASR dan diarization membutuhkan banyak iterasi. Kedua, memilih model LLM yang cukup ringan tapi tetap punya kemampuan multilingual untuk RAG, karena sebagian besar model kecil kurang bagus dalam Bahasa Indonesia.

PENGEMBANGAN

? Apa fitur yang paling ingin Anda tambahkan selanjutnya?

💬 Prioritas pertama adalah real-time streaming transcription berbasis WebSocket — sehingga pengguna bisa melihat transkrip muncul langsung saat meeting berlangsung tanpa harus upload file. Prioritas kedua adalah fine-tuning Whisper untuk Bahasa Indonesia agar akurasi transkripsi meningkat signifikan.

PENGEMBANGAN

? Apakah sudah ada rencana untuk support bahasa daerah Indonesia?

💬 Untuk saat ini fokus utama adalah Bahasa Indonesia dan Inggris karena model MiniLM yang dipakai sudah multilingual untuk 50+ bahasa. Dukungan bahasa daerah seperti Sunda, Jawa, atau Batak memerlukan dataset dan fine-tuning model ASR yang spesifik — ini bisa menjadi roadmap jangka panjang yang sangat menarik.

=====

Tips: Baca setiap jawaban 1–2 kali sebelum presentasi agar terdengar natural, bukan seperti membaca.

PERTANYAAN SEPUTAR PROSES Pengerjaan Project

Tantangan, Proses, dan Pengalaman | MeetRecall AI

TANTANGAN SELAMA Pengerjaan

TANTANGAN

? Apa tantangan terbesar yang kamu hadapi saat membangun MeetRecall AI?

☞ Tantangan terbesar ada di integrasi pipeline Whisper dan Pyannote. Keduanya menghasilkan output dalam format berbeda — Whisper menghasilkan chunks dengan timestamp, sedangkan Pyannote menghasilkan segmen diarization. Menyatukan keduanya secara akurat melalui overlap timestamp membutuhkan banyak iterasi dan debugging. Satu perbedaan kecil dalam cara menghitung overlap bisa mengakibatkan label pembicara yang salah di seluruh transkrip.

TANTANGAN

? Apakah ada kendala teknis spesifik yang sempat bikin stuck cukup lama?

☞ Ya — ada satu masalah yang cukup memakan waktu: model Pyannote adalah gated model di HuggingFace, artinya harus accept terms of service secara manual sebelum bisa didownload lewat kode. Saat pertama kali deploy, backend terus gagal dengan error autentikasi yang tidak jelas. Setelah investigasi, ternyata token HuggingFace sudah benar tapi license model belum di-accept. Itu pelajaran penting soal pentingnya membaca dokumentasi model secara menyeluruh.

TANTANGAN

? Bagaimana kamu mengatasi keterbatasan resource saat menjalankan banyak model sekaligus?

☞ Saya menggunakan pola singleton untuk setiap model — Whisper, Pyannote, embedding model, dan LLM masing-masing hanya di-load satu kali ke memori dan di-share antar request. Pyannote juga saya buat lazy-load, artinya baru di-inisialisasi saat pertama kali ada request transkripsi, bukan saat server start. Ini mengurangi waktu startup secara signifikan dan mencegah OOM error di awal.

TANTANGAN

? Apakah ada bagian yang sempat ingin kamu ganti atau refactor ulang?

☞ Ada. Awalnya saya menggunakan pendekatan synchronous untuk semua pipeline AI, yang membuat server blocking saat memproses file panjang. Saya kemudian refactor ke asyncio dengan `run_in_executor` agar proses berat berjalan di thread terpisah dan server tetap responsif untuk request lain. Ini perubahan arsitektur yang cukup signifikan tapi hasilnya user experience jauh lebih baik.

PROSES & ALUR Pengerjaan

PROSES

? Berapa lama waktu pengerjaan proyek ini dari awal sampai selesai?

☞ Proyek ini dikerjakan dalam waktu sekitar 3–4 minggu. Minggu pertama fokus pada riset model dan arsitektur sistem. Minggu kedua implementasi pipeline transkripsi dan diarization. Minggu ketiga

pengembangan fitur RAG, topic clustering, dan sentiment analysis. Minggu keempat integrasi frontend Streamlit, containerisasi Docker, dan testing end-to-end.

PROSES

? Bagaimana kamu menentukan model-model yang dipakai dalam proyek ini?

💬 Proses pemilihan model berdasarkan tiga kriteria: ukuran model harus bisa berjalan di hardware consumer-grade, kemampuan multilingual karena target user bisa berbahasa Indonesia atau Inggris, dan lisensi open-source. Setiap model dievaluasi secara manual dengan sample audio sebelum diputuskan masuk ke pipeline final. Misalnya, saya mencoba beberapa model embedding sebelum akhirnya memilih paraphrase-multilingual-MiniLM-L12-v2 karena kombinasi ukuran kecil dan akurasi yang baik.

PROSES

? Bagaimana kamu melakukan testing untuk memastikan setiap fitur berjalan benar?

💬 Saya menggunakan rekaman meeting nyata dari YouTube sebagai dataset testing — rekaman berdurasi 24 menit dengan 7 pembicara. Setiap fitur diuji secara terpisah: transkripsi diverifikasi akurasi secara manual, RAG diuji dengan 10+ pertanyaan berbeda, topic clustering dievaluasi apakah topik yang dihasilkan masuk akal, dan sentiment analysis dicek konsistensinya dengan tone percakapan yang jelas. Tidak ada unit test otomatis, pengujian dilakukan secara manual end-to-end.

PROSES

? Apakah kamu mengerjakan ini sendirian? Bagaimana cara kamu mengorganisir pekerjaannya?

💬 Ya, proyek ini saya kerjakan sepenuhnya sendiri sebagai personal project. Untuk organisasi kerja, saya menggunakan pendekatan feature-by-feature — selesaikan satu fitur sampai benar-benar berjalan sebelum lanjut ke berikutnya. Saya juga memisahkan layer dengan jelas: routers untuk HTTP handler, services untuk business logic AI, dan models untuk schema data. Pemisahan ini membuat debugging jauh lebih mudah karena setiap layer bisa diuji secara independen.

PEMBELAJARAN & REFLEKSI

REFLEKSI

? Apa hal terpenting yang kamu pelajari dari proyek ini?

💬 Hal terpenting yang saya pelajari adalah bahwa membangun sistem AI production-ready jauh lebih kompleks dari sekadar membuat model yang akurat. Tantangan sesungguhnya ada di integrasi antar komponen, penanganan error yang graceful, manajemen memori, dan pengalaman pengguna. Sebuah model dengan akurasi 95% tidak berguna jika sistemnya crash saat file terlalu besar atau koneksi tiba-tiba putus.

REFLEKSI

? Kalau bisa mengulang dari awal, apa yang akan kamu lakukan berbeda?

💬 Saya akan lebih awal mendesain schema data yang konsisten antara backend dan frontend. Di awal proyek, ada beberapa inkonsistensi format response yang menyebabkan bug-bug kecil di sisi frontend. Selain itu, saya akan menulis automated test untuk setiap endpoint API sejak awal, bukan mengandalkan pengujian manual yang lebih lambat dan rentan terlewat.

REFLEKSI

? Bagaimana proyek ini berkontribusi pada perkembangan skill kamu sebagai AI Engineer?

💬 Proyek ini adalah pengalaman pertama saya membangun sistem multimodal yang mengintegrasikan speech AI, NLP, dan generative AI dalam satu platform. Saya mendapat pemahaman mendalam tentang speech processing pipeline, cara kerja RAG secara end-to-end dari embedding sampai generation, dan pentingnya deployment considerations seperti containerisasi dan async processing. Ini adalah proyek yang paling komprehensif yang pernah saya kerjakan.

REFLEKSI

? Apakah ada feedback dari orang lain yang sudah mencoba proyek ini?

💬 Proyek ini awalnya dibangun sebagai technical test untuk posisi di NoLimit Indonesia, dan mendapat feedback positif dari tim evaluator terkait kelengkapan fitur dan kualitas kode. Feedback yang paling berkesan adalah bahwa sistem ini menunjukkan kemampuan untuk tidak hanya mengimplementasikan model AI yang sudah ada, tapi juga mengintegrasikannya menjadi produk yang benar-benar berguna dan siap pakai.

=====

Tips: Jawab pertanyaan ini dengan santai dan personal — ini kesempatan untuk menunjukkan proses berpikir dan karakter kamu, bukan hanya kemampuan teknis.